

I2C-Bus: Grundlagen und Praxisbezug zum Projekt O.A.S.I.S.

1. Was ist I2C überhaupt? (Einfach erklärt)

I2C steht für *Inter-Integrated Circuit* (gesprochen: „I-Quadrat-C“ oder englisch „I-Two-C“). Es ist ein Bussystem, das von Philips (heute NXP) entwickelt wurde, um elektronischen Bauteilen auf einer gemeinsamen Leiterplatte das Sprechen miteinander zu ermöglichen. Man kann sich das wie eine Telefon-Konferenzschaltung vorstellen, bei der alle Teilnehmer an derselben Leitung hängen, aber immer nur zwei gezielt miteinander sprechen.

Das Grundprinzip: Controller und Target

- **Der Controller (Master):** Er ist der Chef auf der Leitung (in meinem Fall der FireBeetle 2 ESP32-S3). Er gibt den Takt an, startet jedes Gespräch und bestimmt, wer wann sprechen darf.
- **Das Target (Slave):** Das sind die „Zuhörer“ bzw. Peripheriegeräte (wie der Distanzsensor VL53L0X). Sie warten passiv darauf, dass sie vom Controller über ihre eindeutige Adresse aufgerufen werden.

2. Wie funktioniert die Hardware? (Die zwei Kabel)

Das Geniale an I2C ist, dass man unabhängig von der Anzahl der Geräte fast immer nur zwei Datenleitungen (plus Strom und Masse) braucht. Das spart extrem viele Pins am Mikrocontroller.

Leitung	Langname	Funktion
SCL	Serial Clock	Die Taktleitung. Der Controller schaltet diese Leitung rhythmisch an und aus (wie ein Metronom), damit alle Bauteile wissen, wie schnell die Daten fließen.
SDA	Serial Data	Die Datenleitung. Über dieses Kabel fließen die eigentlichen Informationen (die Nullen und Einsen) hin und her.

Wichtig: Pull-Up-Widerstände

I2C-Leitungen sind „Open-Drain“. Das bedeutet, die Bauteile können die Leitung nur aktiv auf den Boden (0V / Low) ziehen. Damit die Leitung wieder auf 3.3V (High) steigt, wenn niemand etwas sagt, braucht es zwei Widerstände, die die Leitungen sanft nach oben ziehen – die sogenannten Pull-Up-Widerstände.

3. Das Adressierungs-Protokoll (Wer spricht mit wem?)

Da alle Geräte an denselben zwei Kabeln hängen, muss der Controller ein klares Protokoll nutzen, damit sich die Geräte nicht gegenseitig ins Wort fallen:

1. **Start-Bedingung:** Der Controller zieht die Datenleitung (SDA) auf Low, während der Takt (SCL) noch High ist. Das ist das Signal für alle: „Achtung, jetzt gut zuhören!“
2. **Die Adresse senden:** Der Controller schickt eine 7-Bit lange Adresse durch die Leitung. Jedes I2C-Gerät auf der Welt hat eine ab Werk eingebaute Adresse (z. B. besitzt der VL53L0X standardmässig die Hex-Adresse 0x29).
3. **Lese- oder Schreibbit:** Ein achttes Bit entscheidet, ob der Controller Daten an das Gerät schicken (Write) oder vom Gerät abfragen will (Read).
4. **Bestätigung (ACK/NACK):** Das angesprochene Target-Gerät merkt: „Ah, das ist meine Adresse!“ und zieht kurz die SDA-Leitung runter. Das bedeutet *Acknowledge* (Bestätigung) – die Verbindung steht.
5. **Datenübertragung:** Die eigentlichen Bytes werden übertragen.
6. **Stopp-Bedingung:** Der Controller gibt die Leitungen wieder frei. Die Konferenz ist beendet.

4. Konkreter Bezug zum BÜP-Projekt O.A.S.I.S.

Im Projekt O.A.S.I.S. wurde I2C für die Kommunikation zwischen dem Hauptprozessor (FireBeetle 2 ESP32-S3) und den VL53L0X ToF-Distanzsensoren (Time-of-Flight) eingesetzt. Dabei lief nicht alles sofort glatt, was im Abschlussbericht transparent analysiert wurde. Hier sind die wichtigsten Erkenntnisse im Bezug zum Bus:

Das Adressen-Problem bei mehreren Sensoren

Das Projekt war ursprünglich für drei Distanzsensoren ausgelegt. Da aber alle VL53L0X-Sensoren ab Werk exakt dieselbe I2C-Standardadresse (0x29) besitzen, würden am selben Bus alle drei gleichzeitig antworten, wenn der Controller ruft. Das führt zu Fehlern. *Die softwareseitige Lösung im Code:* Über die sogenannten XSHUT-Pins (Shutdown-Leitungen) des Mikrocontrollers wurden die Sensoren nacheinander einzeln „aufgeweckt“. Sobald ein Sensor wach war, wies ihm die Firmware im Setup eine neue, temporäre I2C-Adresse zu (z. B. 0x30, 0x34 oder 0x35). Erst danach wurde der nächste Sensor eingeschaltet. So hatte jeder Sensor im Betrieb seine eigene, einzigartige Nummer.

Die Hardware-Tücke: Das Silkscreen-Problem am FireBeetle 2

Eine der grössten Hürden im BÜP war ein physikalischer Bestückungs- bzw. Beschriftungsfehler auf dem Board des Herstellers DFRobot (Sektion 5.1 des Berichts). Der Aufdruck auf der Platine (Silkscreen) wies falsche Pins für I2C aus. Erst durch das systematische Studium des offiziellen Hardware-Schemas (DFR0975 V1.0) konnte der Fehler identifiziert und die Leitungen für SDA und SCL korrekt gemappt werden.

Stabilität und Reduktion

Aufgrund von Hardware-Defekten an den Bauteilen während der Montage und unerwarteten Problemen bei den Registerverschiebungen wurde das physische System im finalen Prototyp schlussendlich stabil auf einen einzigen, nach vorne gerichteten Hauptsensor heruntergebrochen. Die Logik für das Multi-Sensor-I2C-Mapping bleibt jedoch als wertvolle Software-Architektur im Code hinterlegt und zeigt das tiefe technische Verständnis für Bussysteme.

5. Fazit

I2C ist ein mächtiges, aber fehleranfälliges System. Im Projekt O.A.S.I.S. hat sich gezeigt, dass Theorie (einfaches Anschliessen von zwei Kabeln) und Praxis (Adresskonflikte, falsche Board-Beschriftungen und Signalstabilität) oft weit auseinanderliegen. Die erfolgreiche Fehlersuche und das Verstehen der Adressarchitektur bilden das Kernfundament der elektrotechnischen Arbeit in diesem Projekt.